Logic Control · Arquitectura y Documentación

Sistema de Gestión de Inventarios Multi-Sede

Comercial Norte SRL · Versión actual de Logic Control

Documentación Técnica y Arquitectura Profesional — Logic Control (HTML5 + CSS3 + Vanilla JS)

> **Actualización 2026-10-28 — versión estática sin Node**
>
> Esta revisión eliminó `node_modules/`, `package.json`, `package-lock.json`, `vite.config.js`, `dist/` y el script Node de generación de documentación. `index.html` carga directamente CSS y módulos ES relativos desde `src/`; no hay JSX, imports bare ni dependencias npm.
>
> El header ya no contiene buscador global, lupa, atajo `⌘K` ni botón "Menú". El filtro de Productos permanece en `src/views/ProductsView.js`.
>
> El sidebar reserva un riel de 72 px: inicia colapsado, se expande a 260 px por `:hover` / `:focus-within`, muestra textos/badges durante la expansión y no modifica la posición del contenido. Se eliminó el estado manual `sidebarVisible` / `mobileOpen` y el botón de cierre.
>
> Se eliminó la pantalla y el flujo exclusivo de Proveedores: ruta, ítem lateral, vista, drawer, acciones y servicios de alta/estado. La conversión de kits permanece; su contador se calcula con SKUs `KIT-*` y es independiente del módulo eliminado.

Operación sin Node

Ejecutar desde la raíz con un servidor estático, por ejemplo `python3 -m http.server 8000`, y abrir `http://localhost:8000/`. No ejecutar `npm install`, `npm run` ni Vite: ya no forman parte del proyecto.

Este documento describe en detalle la **arquitectura profesional por capas** implementada para **Logic Control — Sistema de Gestión de Inventarios (Comercial Norte SRL)**, migrado íntegramente a **HTML5 semántico, CSS3 modular y JavaScript nativo (ES Modules)** sin dependencias de frameworks ni backend.

---

1. Visión General de la Arquitectura

Corrección del layout del sidebar — 2026-10-28

Se corrigió el solapamiento del contenido con la navegación lateral. El shell ahora reserva `--sidebar-rail-width: 72px` mediante `padding-left` en `.app-shell`; el contenido comienza después del riel y no queda debajo de él. El sidebar sigue siendo `position: fixed`, por lo que su expansión a `--sidebar-expanded-width: 260px` ocurre como overlay y no desplaza la página.

La implementación quedó centralizada en `src/styles/variables.css` y `src/styles/polish.css`: tokens de ancho/sombra para ambos temas, transición de 220 ms, íconos SVG de 20 px, botones de 40 px, textos truncados con `white-space: nowrap`, badges convertidos en puntos al colapsar, secciones sin espacios vacíos y footer con avatar centrado/logout visible únicamente al expandir. Los botones de navegación mantienen `title` como tooltip nativo en estado colapsado.

La evidencia visual se encuentra en `docs/screenshots/sidebar-before-1280.png` y `docs/screenshots/sidebar-after-1280.png`. La verificación cubre 1280, 1440 y 1920 px, temas claro y oscuro, y todas las pantallas principales; se revisó que breadcrumb/título estén completos, que no exista scroll horizontal y que la expansión no mueva el contenido.

Corrección visual aplicada en esta iteración

Se resolvió la desincronización que impedía que la interfaz se leyera correctamente: el HTML utilizaba clases como `main-area`, `page`, `nav-item` y `panel-header`, mientras que parte del CSS heredado apuntaba a nombres anteriores (`app-main`, `page-content`, `nav-link`). Se incorporó `src/styles/polish.css`, una capa de integración que alinea ambas nomenclaturas, centraliza tokens semánticos (`--text-primary`, `--border`, `--accent`, estados y superficies), ajusta márgenes y agrega los overrides del modo oscuro.

Resultado verificado: aplicación estática servida por HTTP; dashboard probado en modo claro y oscuro.

El proyecto implementa una **Arquitectura Limpia por Capas (Clean Layered Frontend Architecture)** desacoplada en **6 capas especializadas**, utilizando los patrones **Observable Store (Pub/Sub)**, **Service Layer**, **Declarative Template Rendering** y **Centralized Event Delegation**.

```
App_-Web_Inventarios/
├── index.html                          # Punto de montaje HTML5 semántico (#root)
├── README.md                           # Guía de ejecución estática y cambios
├── docs/
│   ├── ARQUITECTURA_Y_DOCUMENTACION.md # Manual técnico de arquitectura y funciones
│   ├── guia-logic-control.pdf           # Informe ejecutivo PDF con capturas y arquitectura
│   ├── documentacion-cambios-frontend.pdf
│   └── screenshots/                    # Evidencia visual de alta resolución (1440×900)
└── src/
    ├── main.js                         # Orquestador principal, render loop y Event
Delegation
    ├── core/                           # Capa 1: Núcleo técnico (Store, Router, Íconos,
Formatters)
    │   ├── store.js
    │   ├── router.js
    │   ├── formatters.js
    │   └── icons.js
    ├── data/                           # Capa 2: Catálogo de datos iniciales y métricas por
sede
    │   └── mockData.js
    ├── services/                       # Capa 3: Lógica de negocio pura e inmutable
    │   ├── inventoryService.js
    │   └── authService.js
    ├── components/                     # Capa 4: Componentes UI reutilizables y Drawers
    │   ├── Sidebar.js
    │   ├── Header.js
```

```
    │   ├── KpiCard.js
    │   ├── StatusBadge.js
    │   ├── DataTable.js
    │   ├── BulkImportDrawer.js
    │   ├── AccountSecurityPanel.js
    │   └── DrawerManager.js
    ├── views/                          # Capa 5: Vistas completas de cada pantalla del
sistema
    │   ├── LoginView.js
    │   ├── DashboardView.js
    │   ├── ProductsView.js
    │   ├── ConversionView.js
    │   ├── MovementsView.js
    │   ├── DirectoriesView.js
    │   ├── ReportsView.js
    │   └── SettingsView.js
    └── styles/                         # Capa 6: Arquitectura CSS3 modular (Design System)
        ├── main.css
        ├── variables.css
        ├── base.css
        ├── layout.css
        ├── components.css
        ├── views.css
        └── polish.css                   # Integración de clases y tokens visuales
```

---

2. ¿Para qué funciona cada capa y cada archivo?

Capa 1 — Núcleo Técnico (`src/core/`)

Contiene la infraestructura transversal de la aplicación. No depende de ninguna vista ni componente.

Archivo	Propósito y Responsabilidad	Funciones Clave y qué hace cada una
`src/core/store.js`	**Estado Global Reactivo (Patrón Observable / Pub-Sub)**. Mantiene una única fuente de verdad (*Single Source of Truth*) para productos, sucursales, movimientos, usuarios, tema visual (`light` / `dark`) y formularios. Sincroniza automáticamente cambios con `localStorage` y `sessionStorage` .	• `store.getState()` : Devuelve el árbol de estado actual. • `store.setState(patch)` : Actualiza el estado de forma parcial o funcional, persiste `theme` y `users` , y dispara el re-render. • `store.subscribe(listener)` : Registra observadores. • `store.showToast(msg)` : Muestra notificaciones flotantes. • `createEmptyBulkRows()` : Genera filas iniciales para la carga masiva.
`src/core/router.js`	**Enrutador SPA Client-Side (History API)**. Controla la navegación entre las pantallas del sistema actualizando la URL (`?screen=...`) sin recargar el navegador.	• `navigate(screen)` : Cambia la pantalla activa. • `navigateFromMenu(screen)` : Cambia de sección desde el sidebar. • `initRouter()` : Escucha el evento `popstate` .
		• `formatNumber(n)` : Formatea cantidades con separador de miles (`es-AR`).

Archivo	Propósito y Responsabilidad	Funciones Clave y qué hace cada una
`src/core/formatters.js`	**Formateadores, Sanitización XSS y Parser CSV**. Estandariza la presentación de números, moneda ARS y cálculo de estados de inventario.	• `formatCurrency(n)` : Formatea importes en Pesos Argentinos (`$ ARS`). • `stockStatus(product)` : Evalúa si un SKU está `'En stock'` , `'Stock bajo'` o `'Sin stock'` comparando `stock` vs `minStock` . • `escapeHtml(value)` : Sanitiza cadenas dinámicas para impedir inyección HTML/XSS. • `parseCsvProducts(text, branch)` : Parsea archivos CSV separados por `,` o `;` y los transforma en objetos de producto.
`src/core/icons.js`	**Motor de Íconos Vectoriales SVG Nativos**. Elimina la necesidad de librerías externas renderizando íconos SVG limpios y el logo multicolor oficial de Google.	• `icon(name, size)` : Retorna el elemento `<svg>` vectorial correspondiente con el tamaño en píxeles indicado.

Capa 2 — Datos de Dominio (`src/data/`)

Archivo	Propósito y Responsabilidad	Estructuras Exportadas
`src/data/mockData.js`	**Catálogo Maestro de Datos Simulados para Comercial Norte SRL**. Provee datos realistas para operar todas las pantallas 100% en el navegador sin backend.	• `initialBranches` , `initialProducts` , `initialKitRecipes` , `initialCategories` , `initialMovements` , `initialUsers` , `googleAccounts` y `chartSeriesByBranch` .

Capa 3 — Servicios de Negocio (`src/services/`)

Encapsula todas las reglas de negocio de inventario y seguridad. Modifica el estado a través de `store.setState` y emite confirmaciones visuales.

Archivo	Propósito y Responsabilidad	Funciones Clave y qué hace cada una
`src/services/inventoryService.js`	**Reglas de Negocio de Almacén, Conversión Mayorista y Sedes**. Ejecuta altas, bajas, modificaciones, desarmes de cajas y movimientos auditados.	• Altas/ediciones/bajas de productos. • Registro de movimientos. • `executeKitConversion()` para descontar bultos, sumar unidades y generar dos movimientos. • Importación masiva, alta de sedes y alta de categorías.
`src/services/authService.js`	**Autenticación, Permisos de Usuarios y Seguridad de Contraseña**. Gestiona la sesión activa, el estado habilitado/inhabilitado de operadores y la política de claves.	• `loginSession(msg)` / `logoutSession()` : Inicia o cierra la sesión en `sessionStorage` . • `toggleUserEnabled(userId)` : **Habilita o inhabilita** en tiempo real el acceso de un usuario. • `updateUserRole(userId, role)` : Cambia el rol operativo de un usuario. • `createNewUser()` : Da de alta un nuevo operador asignado a una sede. • `evaluatePasswordStrength(pwd)` : Calcula el puntaje de seguridad (0 a 4) verificando longitud, mayúsculas, números y símbolos. • `submitPasswordChange()` : Valida la clave

Archivo	Propósito y Responsabilidad	Funciones Clave y qué hace cada una
		actual, la longitud mínima y la coincidencia antes de actualizar la contraseña.

Capa 4 — Componentes UI Reutilizables (`src/components/`)

Funciones puras que reciben el `state` (o `props`) y devuelven fragmentos HTML semánticos.

Archivo	Propósito y Responsabilidad	Qué implementa
`src/components/Sidebar.js`	**Sidebar de navegación dinámica**.	• Íconos y contadores dinámicos. • Riel reservado de 72 px y expansión a 260 px por hover/foco como overlay. • Selector claro/oscuro y avatar/logout alineados.
`src/components/Header.js`	**Barra Superior (Topbar)**.	• Selector de Sedes/Sucursales. • Cambio rápido de tema, alertas, carga masiva y alta de producto.
`src/components/BulkImportDrawer.js`	**Drawer de Carga Masiva de Productos**.	• **Pestaña 1 (Tabla / Formulario interactivo)**: Grilla editable en vivo para cargar o pegar múltiples productos indicando nombre, SKU, categoría, sede, stock, mínimo, costo, precio y ubicación, con cálculo en vivo de unidades y valorización. • **Pestaña 2 (Importar CSV/Excel)**: Lector de archivos `.csv` con previsualización y descarga de plantilla modelo.
`src/components/AccountSecurityPanel.js`	**Panel de Control de Usuarios y Cambio de Contraseña**.	• Lista de usuarios con estado `Habilitado` / `Inhabilitado`, selector de rol y botón directo para **Habilitar / Inhabilitar** acceso. • Formulario desplegable para dar de alta nuevos operadores. • Formulario de **Cambio de Contraseña** con visor mostrar/ocultar y medidor visual de fortaleza de 4 barras.
`src/components/DrawerManager.js`	**Orquestador de Ventanas Laterales (Drawers)**.	• Renderiza los drawers de Producto, Movimiento, Categoría, Sedes/Sucursales y Carga Masiva.
`src/components/DataTable.js`	**Tabla de Datos Genérica**.	• Renderiza encabezados, celdas alineadas y estado vacío con acción de restablecimiento.
`src/components/KpiCard.js`	**Tarjeta de Indicador Clave (KPI)**.	• Muestra métrica principal con tipografía tabular, variación porcentual y nota al pie.
`src/components/StatusBadge.js`	**Etiqueta Semántica de Estado**.	• Renderiza badges codificados por color (`En stock`, `Stock bajo`, `Sin stock`, `Ingreso`, `Egreso`, `Ajuste`, `Transferencia`).

Capa 5 — Vistas de Pantalla (`src/views/`)

Archivo	Pantalla	Funcionalidad Principal
`src/views/LoginView.js`	**Login Corporativo**	Botón oficial **Continuar con Google Workspace (SSO)** con logo multicolor, selector interactivo de cuentas corporativas verificadas y formulario tradicional por correo/clave.
`src/views/DashboardView.js`	**Dashboard Ejecutivo Multi-Sede**	Muestra por defecto el **resumen consolidado de Todas las sedes** (KPIs sumados y tarjetas comparativas por sucursal) e incluye una **barra de filtro por sede** (`Todas las sedes` , `CD Norte` , `Sucursal Centro` , `Depósito Sur`) que recalcula KPIs, gráficos, actividad reciente y tabla de existencias para cada planta individual.
`src/views/ProductsView.js`	**Catálogo de Productos**	Búsqueda instantánea, filtros por categoría y estado de stock, ordenamiento dinámico, paginación, exportación CSV y edición/eliminación por fila.
`src/views/ConversionView.js`	**Convertir Kits / Cajas**	Selección de recetas de empaque o modo libre, simulador de impacto en stock (`-N cajas` → `+M unidades`) y confirmación con doble asiento en movimientos.
`src/views/MovementsView.js`	**Historial de Movimientos**	Bitácora filtrable por tipo de operación (`Ingreso` , `Egreso` , `Ajuste` , `Transferencia`) y buscador por SKU, remito o responsable.
`src/views/DirectoriesView.js`	**Categorías y ubicaciones**	Tarjetas de rubros con margen objetivo y valorización.
`src/views/ReportsView.js`	**Reportes y Analítica**	Capital inmovilizado a costo, valor de realización a precio de lista, participación porcentual por rubro y tabla de prioridad de Orden de Compra sugerida.
`src/views/SettingsView.js`	**Configuración del Sistema**	Datos fiscales de la empresa, reglas automáticas de almacén, **Control de Usuarios (Habilitar/Inhabilitar)** y **Cambio de Contraseña**.

Capa 6 — Arquitectura CSS3 Modular (`src/styles/`)

El diseño visual está dividido en 7 hojas de estilo puras importadas desde `src/styles/main.css` :

1. `src/styles/variables.css` : Tokens de diseño (CSS Custom Properties) para **Modo Claro (`:root`)** y **Modo Oscuro (`[data-theme='dark']`)**, tipografía (`Inter` y `JetBrains Mono`), radios, sombras y paleta semántica.
2. `src/styles/base.css` : Reset moderno, normalización de controles, tipografía tabular (`.tabular`) y foco accesible (`:focus-visible`).
3. `src/styles/layout.css` : Grid principal `.app-shell` con soporte para **menú auto-ocultable (`.sidebar-hidden` / `.sidebar-visible`)**, `.sidebar` , `.topbar` , selector de tema debajo de Configuración y menú desplegable de sucursales.
4. `src/styles/components.css` : Botones (`.btn` , `.btn-primary` , `.btn-secondary`), inputs, KPI cards, tablas de datos, badges, drawers modales y barra de notificaciones toast.
5. `src/styles/views.css` : Estilos específicos para el Login con Google Workspace, tarjetas comparativas de sedes en el Dashboard, módulo de conversión de kits/cajas, tabla editable de carga masiva y paneles de seguridad/usuarios en Configuración.
6. `src/styles/main.css` : Punto de entrada que ensambla las hojas en orden de cascada.

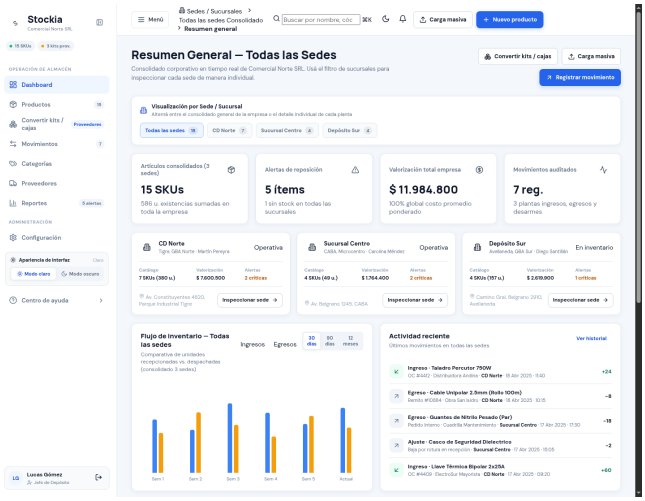
7. `src/styles/polish.css` : Capa de integración que corrige la lectura de clases entre HTML y CSS, normaliza espaciado, botones, paneles, dashboard responsive y contraste claro/oscuro.

Orquestador Principal (`src/main.js`)

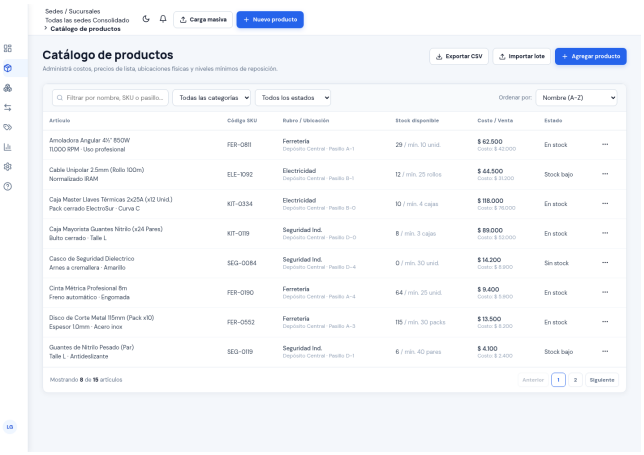
El archivo `src/main.js` conecta todas las capas mediante tres mecanismos profesionales:

1. **Ciclo de Renderizado con Preservación de Foco (`renderApp` y `restoreFocus`)**: Cada vez que `store.setState()` notifica un cambio, `renderApp()` captura el elemento activo y la posición del cursor (`selectionStart` / `selectionEnd`), actualiza el HTML declarativo y restaura el foco de forma transparente para que el usuario pueda escribir de corrido en buscadores y formularios.
2. **Delegación Centralizada de Eventos (`Event Delegation`)**: En lugar de adjuntar cientos de listeners individuales por botón, se registran listeners únicos en `#root` para `click` , `input` , `change` y `submit` utilizando atributos semánticos `data-action` , `data-input` , `data-draft` , `data-bulk-row` y `data-setting-input` .
3. **Atajo global de teclado**: `Escape` cierra cualquier drawer o menú desplegable abierto.

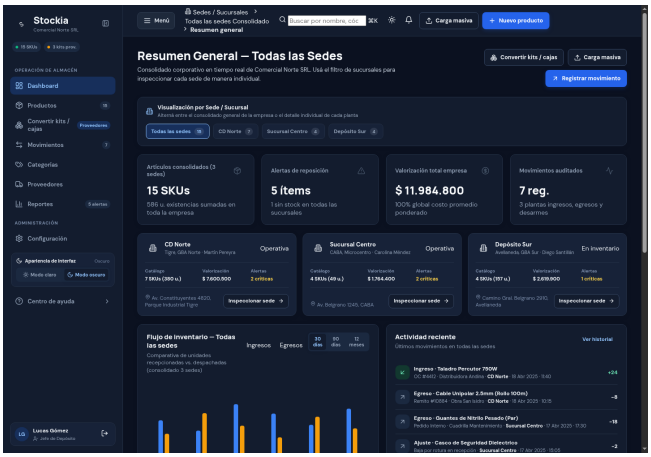
Evidencia visual



02-dashboard-consolidado



logic-control-productos-1440-after



06-configuracion-dark-mode